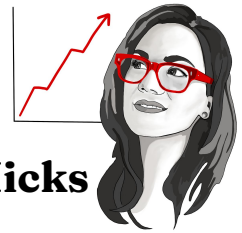
Protect your problem-solving in the era of agentic coding

The view from learning science



Cat Hicks

Catharsis *for* tailscale

Why care about this?

It wins.

Individual: Adaptive learning is incredibly efficient

Group: Groups with self-corrective processes outperform solo genius

Society: Expertise-formation strategies drive up high achievement

Tech: Knowledge work is increasingly reliant on group work; too big for any single brain; marked by adaptive learning on others' solutions

Species/planet: Our cognition optimizes for *social learning* and *teaching* cycles; this is *cumulative culture* and it's a species-level survival trait

See sources at: github.com/DrCatHicks/tailscale_talk

What informs my POV on software teams

- PhD in Psychological Sciences ("lab not couch"), studied performance + disclosure + how social cognition shapes achievement inferences
- I care about things that are **tractable.** I think a lot about: *within-person* change (more than between-person differences), the environments that shape the possibilities we see, and how beliefs & appraisals shape our choices

Lee, C. S., & Hicks, C. M. (2024). Understanding and effectively mitigating code review anxiety. *Empirical Software Engineering*, *29*(6), 161.

What are you thinking about most when you think about learning, problem-solving and AI?

One of the things I think about:

Our internal theories about *technical ability, intelligence,* and *identity* can deeply impact our wellbeing and change what we use as cues for our *behavior*

The first open science I did for developers was about “learning debt”!

PsyArXiv **“It’s Like Coding In The Dark”: The Need For Learning Cultures Within Coding Teams**

Page: 2 of 26 Automatic Zoom

EXECUTIVE SUMMARY

- This report presents data from a qualitative research project with software engineers and developers. Twenty-five full-time code writers completed a “debugging” task and an in-depth interview on their learning, problem-solving, and feedback experiences while onboarding to an unfamiliar, collaborative codebase. This report applies a learning science lens to inform an understanding of how to help code writers can thrive and collaborate.
- Themes from these interviews revealed an important learning tension: the work that code writers needed to do to *understand* code often did not feel like what was rewarded in the *evaluation* of their work. Code review often did not recognize code writers’ effort when it did not result in lines of code. Despite stated ideals about knowledge sharing (e.g., documentation and collaboration), this work was often contradicted with negative cues from colleagues about what was “truly” valued. This tension was exacerbated by code writers’ fears about “not looking like an engineer,” and their desire to perform to the expectations of their environments.
- Code writers navigated this by divesting from their own learning and from the “invisible” work of knowledge transfer, leaving future collaborators without guidance in their own ramp-up to unfamiliar code. As a result, code writers frequently expressed a poignant loneliness, even in highly resourced teams.
- This maladaptive cycle can be understood as *Learning Debt*. Research from learning science describes how environments that discourage sharing mistakes and valuing “in-draft” effort lead to long-term costs in people’s motivation, wellbeing, and learning. Under this discouragement, even formal processes which are ostensibly meant to maintain productivity and provide support (e.g., code review, conversations with senior code authors) can reinforce these negative norms.

Authors

Catherine M. Hicks

Abstract

This report presents data from a qualitati
project with software engineers and...

Read more

Public Data

Because this was a qualitative interview p
participants opted only to share their resp
analyzed in the report, further data such a
transcripts and information about particip
available.

Public Preregistration

Conflict Of Interest

Author asserted no Conflict of Interest.

Conflict Of Interest

Author asserted no Conflict of Interest.

One big takeaway:

When learning feels like it isn’t a valued & shared part of tech **identity**, people hide a lot of their deep technical problem-solving & we get cultures that don’t efficiently solve together

Catharsis Consulting

Mostly True? or **Mostly False?**

1. Your head is an empty bucket that you fill

2. Performance = learning (linear relationship)

More than mere exposure:

Expertise means building *structural* understanding

#1: “Your head is an empty bucket that you fill”

This is the tape recorder myth

→ Actually, encoding is selective. Experts sort by structural features and build mental maps and deep schemas; beginners sort by surface features without deep schema

Retrieval is generative (we used this word before AI!)

→ conditions that produce forgetting *enhance* learning, and successful retrieval prompts learning. This is because pulling information into your mind is more reconstructive (even when retrieval fails!) than it is “pulling information from storage”

Structural understanding helps us switch between drill-down problem-solving & offloading

→ we are constantly building/losing surface-level skills, but structural understanding helps us choose the right ones & efficiently upskill

Agentic coding practice: *Be a creator. Generate in Pre-testing & Retrieval*

The Pre-testing effect: Manually generate a small example of the work *before* you scale it out through automation/AI (will even improve how deeply you encode/process the content you review from AI)

The Generation effect: Write down a prediction of a mechanism or effect and see if it holds (even high level guesses will help)

You remember what you attend to and try to retrieve: Practice deliberate retrieval-based learning across your own “ownership” areas of the codebase or mechanisms you want to understand: like driving without GPS. You will retain more of what you actively retrieve

Agentic coding practice: *Be a creator. Generate in Pre-testing & Retrieval*

Can we do this even while taking advantage of AI offloading? I think so

Manually generating a small example of the work *before* you scale it out through automation/AI → **back-and-forth modification** with AI after **you** come up with ideas first might produce higher creativity than *either* AI first or no-AI

Efficient entrypoints → starting "further forward" with **context-personalized worked examples**. Using AI as "wayfinding" support to identify starting places in complex code but then doing active work once you're there

Some interesting freedom & benefit of shifting dev effort toward design, planning & experimenting: **effort-reward** calculations are changing, that changes the **expectancy-value** judgment that drives motivation

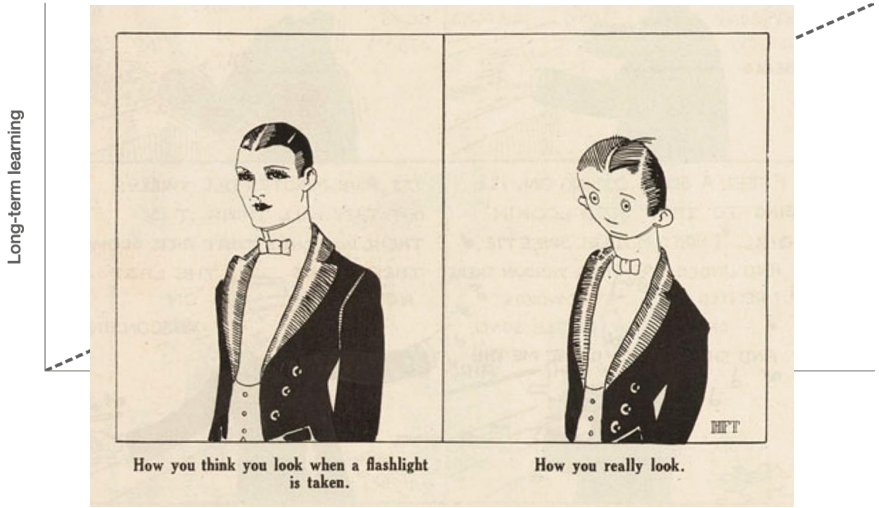
Mostly True? or **Mostly False?**

1. Your head is an empty bucket that you fill

2. Performance = learning (linear relationship)

Lasting learning usually looks *worse* before it looks *better*

#2: “Performance = Learning”



Actually, conditions that promote long-term performance often depress short-term performance!

You want to be doing ***deliberate practice*** more than you’re ***performing***

Core concept: **desirable difficulties**

- Spacing content vs cramming
- Interleaving content
- Retrieval practice

All dampen immediate performance, all lead to better long-term learning!

Illusion of Explanatory Depth: *thinking you understand a complex system more than you do*

A classic programming thinking challenge!

Remember a time when you could follow code line-by-line but struggled to know why it broke when a dependency changed

We *specifically* struggle with this illusion for **causal/mechanistic understanding ! Software code & its systems = complex causal patterns.**

It is fixable. Metacognition challenges this illusion; active learning punctures this illusion. Boundary conditions, irregularities, and interactions between components can all provide clues we have the wrong mental model

Illusion of Fluency: *familiarity or ease makes us think we understand something when we really don't*

"Frictionless" is a mental trap.....

....but *so is "frictionmaxxing"!*
Grinding doesn't win in the long term. People choose maladaptive inefficient strategies often because they fixate strongly on effort & exhaustion cues.

Deliberate practice = outside of comfort zone, getting immediate feedback & end states, breaking skills into components, and *rewarded, not burning out.*

Mere repetition is inefficient; with structured skill-building we can learn more in a shorter amount of time. Think about **what your own skill inventory** would be – and force yourself to get specific and real. These aren't Pokémon; "catch 'em all" is a paralyzing anxiety trap

Give yourself tools to assess what you don’t know and what you do.

Agentic coding practice: *Be an architect of your understanding. Test solutions across contexts, boundaries, and interactions.*

Friction isn't always a negative signal. Test your developing understanding on a longer time period than one session or one day; our immediate perception of fluency and understanding is often inaccurate

Context & boundaries: when you generate AI solutions, look across structurally-similar problems in a completely different domain. Create boundary cases: when would this be the wrong solution?

Map comprehension. Keep track of your codebase comprehension & concept areas you haven't visited in a while. Close the AI output and try to regenerate it from memory

Think about AI as a tool to scaffold challenging work, not a tool to remove effort. But **small effort counts**: even 10-15 minutes a day of learning is effective at scale over time! Deep concept knowledge > syntax taxes

Principle across all of this: active learning that builds your metacognition

Before: designing context, task scoping, goal clarity. Pre-testing activates deeper thinking

During: vary what you work on, intersperse self-testing, own your expertise development, compare across contexts to extract structural features, identify boundaries

After: reconstruct the logic, use time saved to build your codebase comprehension, track longer-term outcomes, trace system connections, flexibly test solutions in new domains

Hot take #1: less a “cognition” problem and more a *metacognition* problem

There are more cognitive options than “decline” or “damage”: own your problem-solving and protect it

Problem-solving is a process, not a single internal attribute:

We harness **diverse abilities** in a **longitudinal process** that interacts with others’ solutions via **social learning.** We can **scaffold** individual characteristics with **metacognitive strategies;** these are often more predictive of long-term outcomes than individual characteristics!

Most people want meaningful engagement; when they don’t, we often find environmental explanations.

Hot take #2: I do not believe succeeding in the AI era means you need to lose the technical problem-solving you love

We will need structural problem-solving to create & maintain technology